NURobotics Firmware Course

Week 5: State Machines + RTOSs

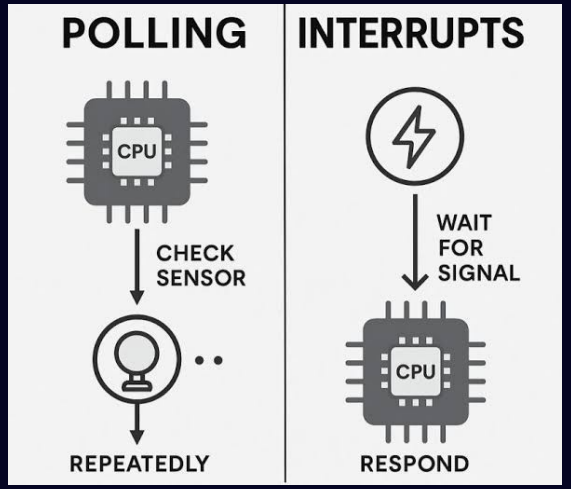
Course Overview

- Week 1: Intro to Firmware & Programming
- Week 2: Analog I/O & Interrupts
- Week 3: Serial Communication Protocols
- Week 4: Driver Development
- **Week 5: State Machines + RTOSs**
- Week 6: STM32 Fundamentals
- Week 7: STM32 Peripherals
- Week 8: Communication and RTOS Capstone

Miscellaneous Topics

Polling vs Interrupts

Polling means to regularly and repeatedly check for a change. Interrupts are triggered when some input changes and are only handled on this change.



- Polling
  - Constantly checks even when not needed
  - Energy intensive
  - Updates information at a fixed time
  - Can check pretty much anything
- Interrupts
  - Only occurs when needed
  - Lower energy cost
  - Requires a specific change condition to occur. This is limited to what your MCU can detect

Watchdog Timers

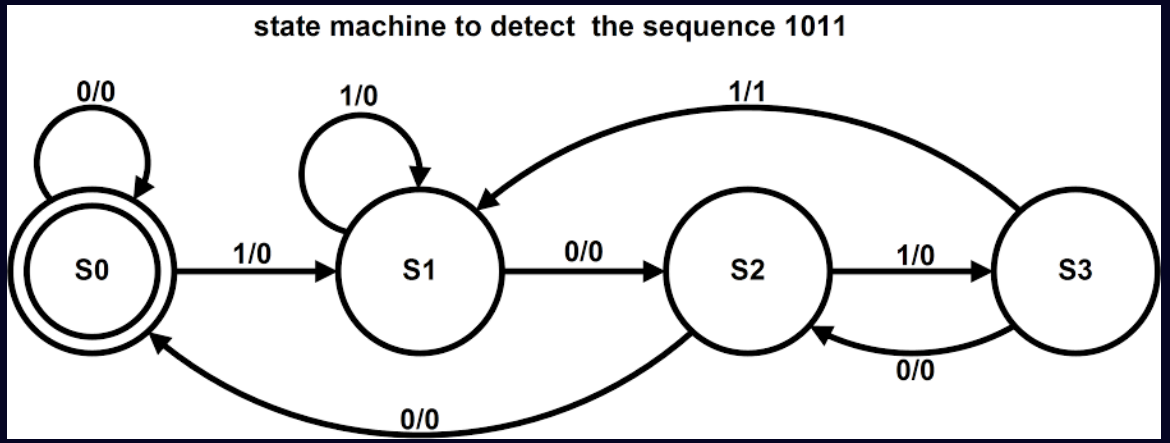
- **Purpose:** Monitor a system or process and detect when it stops operating correctly
  - A watchdog expects a periodic signal within a defined time window
  - If the expected activity is missed, the watchdog triggers a predefined response
- **Internal Watchdogs (MCU/HAL)**
  - Often built into the microcontroller or supported by the HAL
  - Typically monitor whether software periodically refreshes the watchdog timer
  - Often trigger a microcontroller reset if the timeout expires
- **External Watchdogs**
  - Monitor a signal from the system such as a periodic pulse/heartbeat
  - Check that the signal arrives within the configured timing window
  - Can trigger a reset, raise a fault, or initiate another recovery action depending on the device

State Machines

States and Transitions

A state machine is a graph of states and transitions between them

- State
  - A progress point
- Start state (Reset state)
  - The entry point to the state machine
- Transition
  - A decision on when/if to change states



State Transition Table

State transition tables are a text-based formatting of state machines

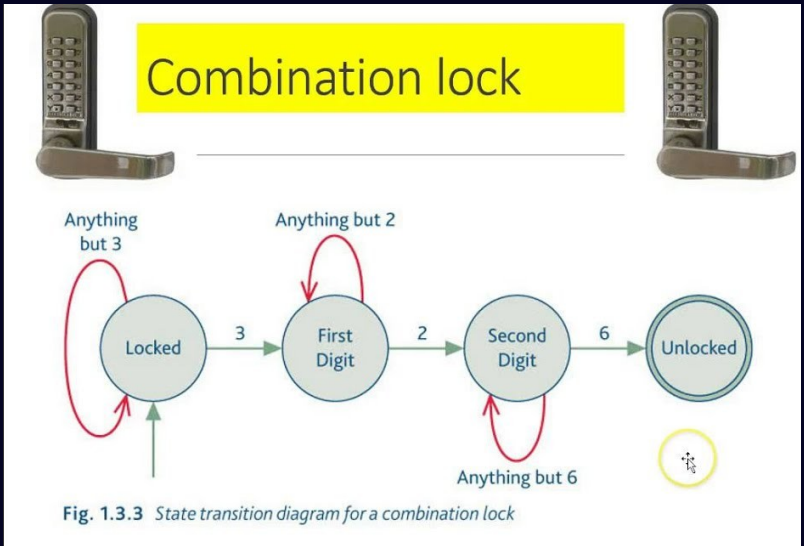


Fig. 1.3.3 *State transition diagram for a combination lock*

s	a	s’
Locked	Anything but 3	Locked
Locked	3	First Digit
First Digit	Anything but 2	First Digit
First Digit	2	Second Digit
Second Digit	Anything but 6	Second Digit
Second Digit	6	Unlocked

Applications

Almost all firmware more complicated than reading a single sensor requires a system of states. Even at its simplest, startup -> main program -> main program etc is the minimum state machine.

However, proper state machines are vital to applications where different tasks or actions need to be done depending on the state of the device.

Overall, this greatly simplifies control flow.

```
#include <Arduino.h>

// Enum classes can have their sizes specified
// and are guaranteed unique
enum class RobotState : uint8_t {
  Idle = 'I',
  Main = 'M',
  Test = 'T',
  Error = 'E'
};

bool startup_tasks();
RobotState idle_task();
RobotState control_loop();
RobotState test_suite();
RobotState error_handler();

RobotState state = RobotState::Idle;

void setup() {
  if (!startup_tasks()) {
    state = RobotState::Error;
  }
}

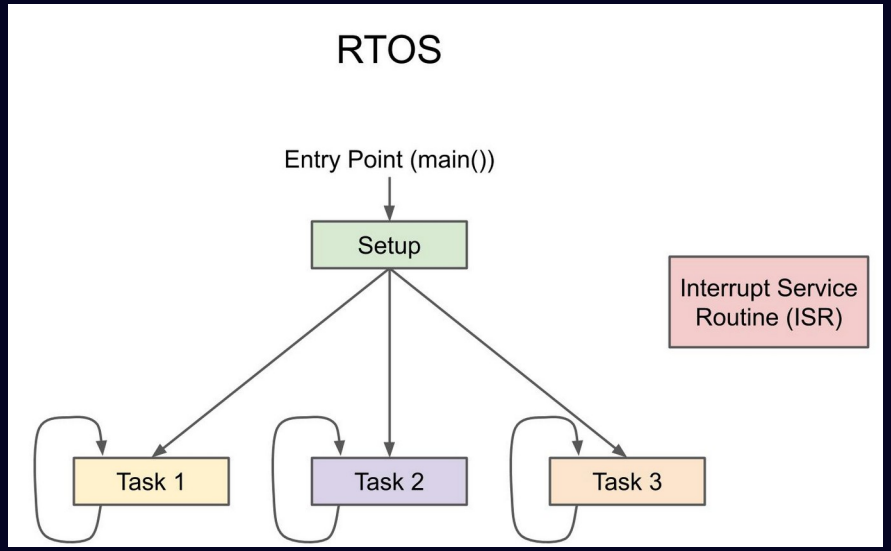
void loop() {
  switch (state) {
    case RobotState::Idle:
      state = idle_task();
    break;
    case RobotState::Main:
      state = control_loop();
    break;
    case RobotState::Test:
      state = test_suit();
    break;
    default:
      state = error_handler();
    break;
  }
}
```

Real Time Operating Systems (RTOS)

What is an RTOS?

An RTOS is an incredibly simplified operating system designed for embedded applications and mostly just managed task threads.

- Much stricter timing constraints than traditional Oss
  - While minor latencies are acceptable in a normal computer, embedded applications require much faster reaction times to stimuli
- Only one process
  - While traditional OSs have to balance many processes, we only ever work with one process
- Much less overhead
  - Scheduling, memory management, and other additional tasks in traditional OSs take up a lot of CPU time and memory



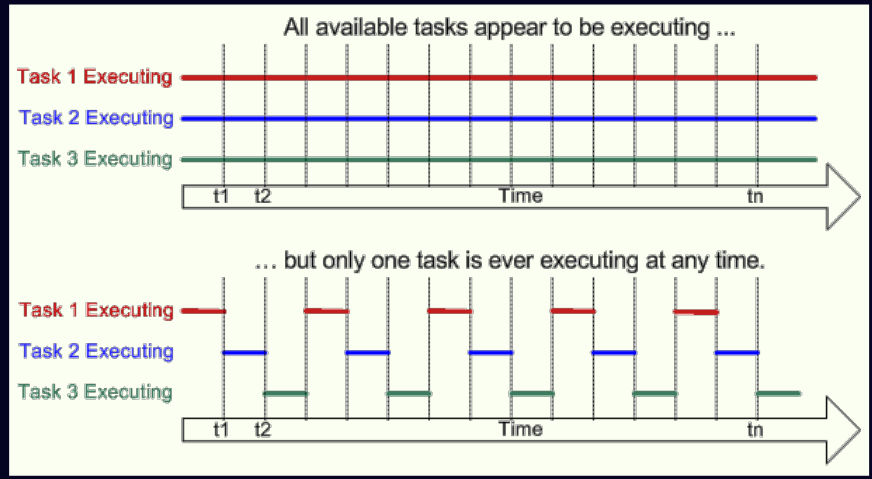
Threads

Threads are just a section of code, like a function. However, unlike normal inline code, multiple threads can be running at once in parallel (concurrency). This works by having a scheduler manage these threads, giving them CPU time and other resources as needed and available.

Even if you only have 1 CPU, an RTOS can make it appear as though you have more.

Threads vs Processes

- Threads
  - Subset of a process
  - Shared memory
- Processes
  - Independent program execution
  - Separate virtual memory



Shared Resources

Because threads can be scheduled non-deterministically, this can result in shared resources being accessed non-deterministically. This can lead to a “data race” (race condition).

A data race occurs when two or more threads try to access a shared memory location with at least one of those accesses being a write.

```
uint8_t x = 0;
uint8_t y = 10;

void t1() {
    x += 5;
    y = x;
}

void t2() {
    x = y;
    x++;
}
```

```
#include <cstdint>
#include <mutex>

uint8_t x = 0;
uint8_t y = 10;

std::mutex mtx;

void t1() {
    mtx.lock();

    x += 5;
    y = x;

    mtx.unlock();
}

void t2() {
    mtx.lock();

    x = y;
    x++;

    mtx.unlock();
}
```

Tasks

The key benefit of an RTOS is the task scheduler. Instead of managing complex state machines, manual timers, and the various priorities of the jobs you need done, you just make tasks.

```
void setup() {
  xTaskCreate(
    toggleLED,    // Function that should be called
    "Toggle LED",   // Name of the task (for debugging)
    1000,            // Stack size (bytes)
    NULL,            // Parameter to pass
    1,               // Task priority
    NULL             // Task handle
  );
}
```

```
void toggleLED(void * parameter){
  for(;;){ // infinite loop

    // Turn the LED on
    digitalWrite(led1, HIGH);

    // Pause the task for 500ms
    vTaskDelay(500 / portTICK_PERIOD_MS);

    // Turn the LED off
    digitalWrite(led1, LOW);

    // Pause the task again for 500ms
    vTaskDelay(500 / portTICK_PERIOD_MS);
  }
}
```

Lab!

Today’s Lab

- State Machine
  - Create a digital lock that can be unlocked using the two buttons
  - Use a state machine to accomplish this